Computer Networks

Overview of the Internet
Protocol layering (Book 1 Ch 1)

Application Layer

- Application-Layer Paradigms
- Client-server applications
  - World Wide Web and HTTP
  - FTP
  - Electronic Mail
  - DNS
- Peer-to-peer paradigm
  - P2P Networks
- Case study: BitTorrent (Book 1 Ch 2)

Application Layer

1. **Introduction**
   - Providing Services
   - Application-Layer Paradigms
2. **Standard Client-Server Applications**
   - World Wide Web and HTTP
   - FTP
   - Electronic Mail
   - Domain Name System (DNS)
3. **Peer-to-Peer Paradigm**
   - P2P Networks

World Wide Web and HTTP

- Section introduces:
  - World Wide Web (WWW)
  - HyperText Transfer Protocol (HTTP)
- WWW is also called:
  - Web
- HTTP is:
  - Most common client-server application protocol
- HTTP used for:
  - Communication on the Web
- Web and HTTP together enable:
  - Access to Internet resources
  - Exchange of web documents

World Wide Web (WWW)

- Web idea proposed by:
  - Tim Berners-Lee
- Proposed in:
  - 1989
- Proposed at:
  - CERN
  - European Organization for Nuclear Research
- Original purpose:
  - Share research among scientists in Europe
- Commercial Web started:
  - Early 1990s

Concept of the Web

- Web is:
  - Repository of information
- Information stored as:
  - Web pages
- Web pages distributed:
  - Across the world
- Related web pages are:
  - Linked together
- Two important concepts:
  - Distributed
  - Linked

Distributed Nature of the Web

- Distribution enables:
  - Growth of the Web
- Any web server can:
  - Add new web pages
- New pages can be:
  - Announced to Internet users
- Distribution avoids:
  - Overloading a few servers
- Web content stored at:
  - Multiple global locations

Linked Nature of the Web

- Web pages can:
  - Refer to other web pages
- Linked pages may exist:
  - On same server
  - On remote servers
- Linking achieved using:
  - Hypertext
- Hypertext concept existed:
  - Before the Internet
- Clicking a link retrieves:
  - Related document automatically

From Hypertext to Hypermedia

- Hypertext originally meant:
  - Linked text documents
- Modern term:
  - Hypermedia
- Hypermedia supports:
  - Text
  - Images
  - Audio
  - Video
- Web pages can contain:
  - Multiple media types

Modern Uses of the Web

- Web originally used for:
  - Retrieving linked documents
- Modern Web supports:
  - Electronic shopping
  - Online gaming
  - Internet radio
  - Television streaming
- Users can access content:
  - Anytime on demand
- No need to:
  - Follow broadcast schedules

Architecture of the WWW

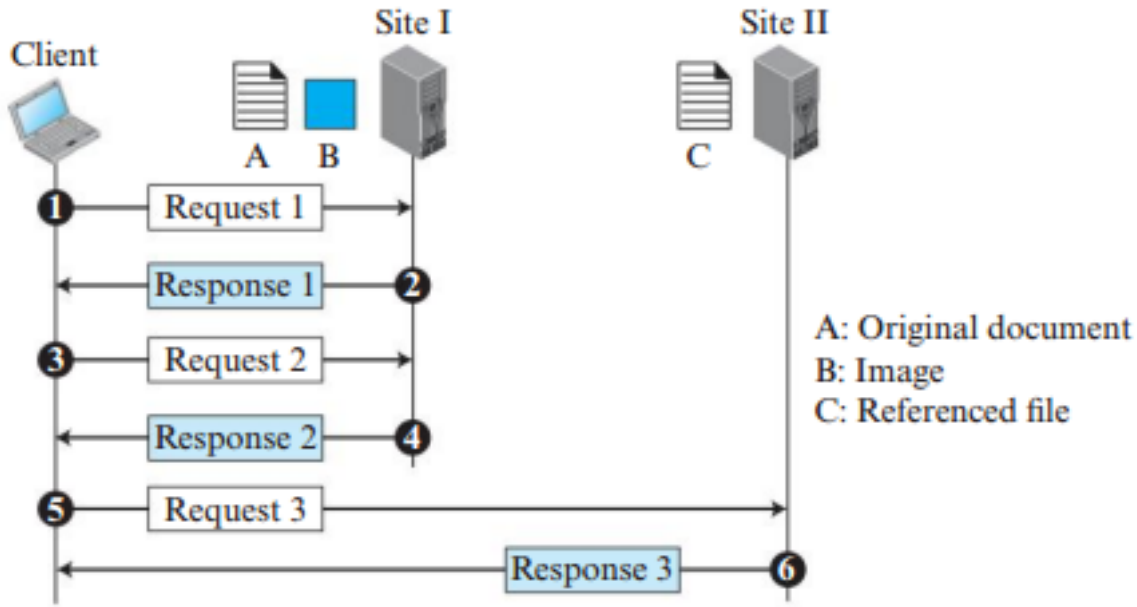
- WWW uses:
  - Distributed client-server architecture
- Client accesses service using:
  - Browser
- Service provided by:
  - Web server
- Services distributed across:
  - Multiple sites
- Each site stores:
  - One or more web pages

Types of Web Pages

- Each web page is:
  - A file
- Each file has:
  - Name
  - Address
- Web pages may contain:
  - Links to other pages
- Two types of web pages:
  - Simple web page
  - Composite web page
- Simple web page:
  - No links
- Composite web page:
  - One or more links

Example

Figure 2.8 *Example 2.2*



Example 2.2: Retrieving Web Documents

- Scientific document contains:
  - Reference to another text file
  - Reference to a large image
- Three files involved:
  - File A
  - File B
  - File C
- File locations:
  - File A and File B stored in Site I
  - File C stored in another site

Transaction 1: Retrieving Main Document

- First transaction retrieves:
  - File A
- File A contains:
  - Reference to image file
  - Reference to text file
- References act as:
  - Pointers to other files
- Retrieved copy allows user to:
  - Browse the main document

Transaction 2: Retrieving Image File

- User clicks:
  - Reference to image
- Second transaction retrieves:
  - File B
- File B contains:
  - Large image
- Although File B stored in same site:
  - Separate transaction required

Transaction 3: Retrieving Referenced Text File

- User clicks:
  - Reference to text file
- Third transaction retrieves:
  - File C
- File C stored:
  - In another site
- Independent retrieval performed through:
  - Separate request/response transaction

Independence of Web Pages

- File A, File B, and File C are:
  - Independent web pages
- Each file has:
  - Independent name
  - Independent address
- References inside File A do not mean:
  - Files B and C depend on File A
- Files can be retrieved:
  - Independently by other users

Example 2.2 Illustration

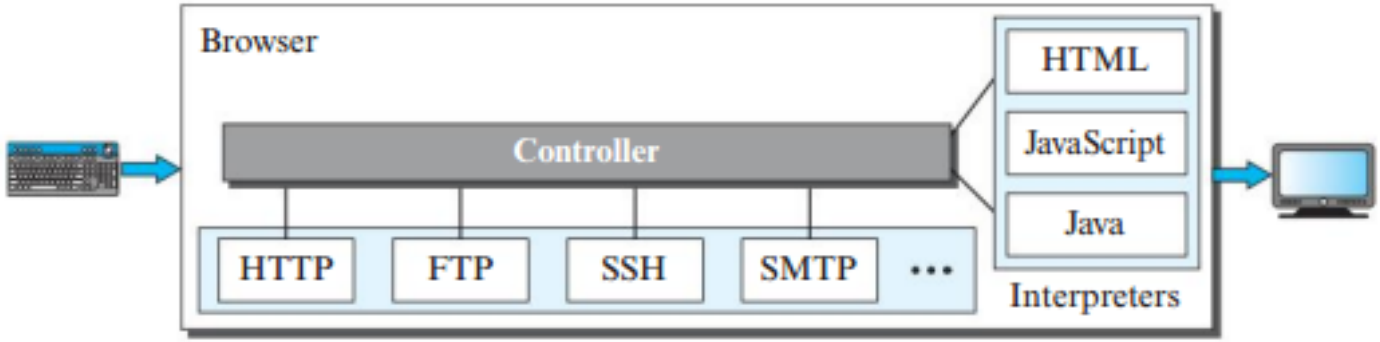
File A (Main Document)
↓ Reference to Image
File B (Image File)
File A (Main Document)

↓ Reference to Text File
File C (Referenced Text File)

- Transaction 1 retrieves File A
- Transaction 2 retrieves File B
- Transaction 3 retrieves File C

Browser

Figure 2.9 *Browser*



Web Client (Browser)

- Web browsers:
  - Interpret web pages
  - Display web pages
- Different vendors provide:
  - Commercial browsers
- Most browsers use:
  - Similar architecture
- Browser consists of:
  - Controller
  - Client protocols
  - Interpreters

Components of a Web Browser

- Controller:
  - Receives keyboard input
  - Receives mouse input
  - Accesses documents using client protocols
- Client Protocols:
  - Communicate with web servers
- Interpreters:
  - Display retrieved documents

Browser Operation

- User interaction handled by:
  - Controller
- Controller uses:
  - Client protocol to retrieve document
- Retrieved document processed by:
  - Interpreter
- Interpreter displays:
  - Content on screen

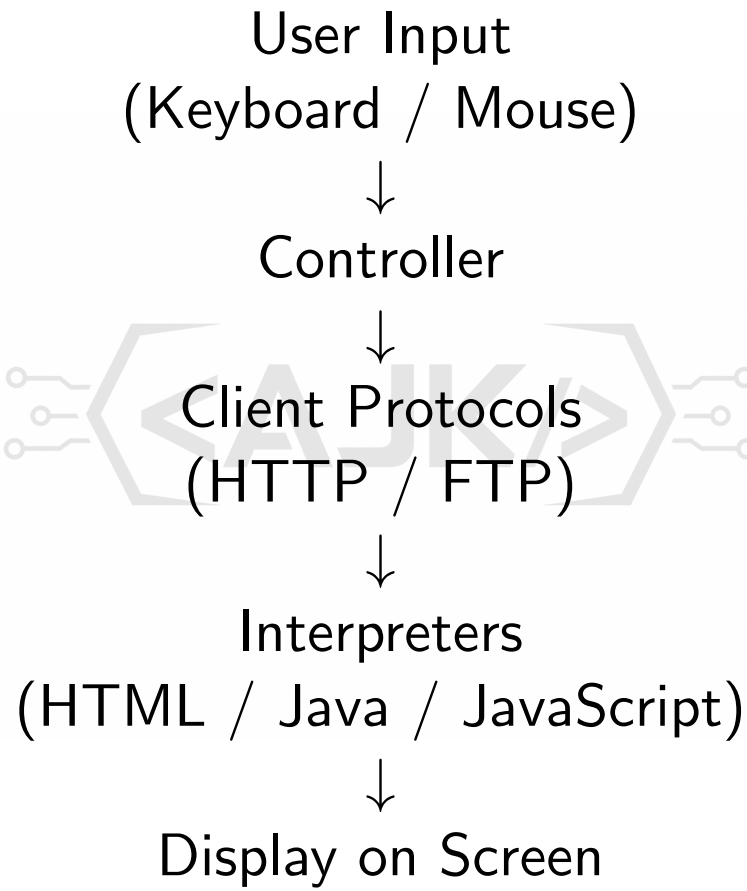
Protocols and Interpreters in Browsers

- Client protocols may include:
  - HTTP
  - FTP
- Interpreter type depends on:
  - Document format
- Common interpreters:
  - HTML
  - Java
  - JavaScript

Popular Web Browsers

- Commercial browsers include:
  - Internet Explorer
  - Netscape Navigator
  - Firefox
- Browsers provide:
  - User-friendly access to the Web

Architecture of a Web Browser



Web Server

- Web pages stored at:
  - Web server
- Server responds to:
  - Client requests
- Requested document sent to:
  - Client browser
- Server provides:
  - Web services

Efficiency Techniques in Web Servers

- Servers improve efficiency using:
  - Cache memory
- Frequently requested files stored in:
  - Main memory
- Memory access faster than:
  - Disk access
- Servers may use:
  - Multithreading
  - Multiprocessing
- Allows server to:
  - Handle multiple requests simultaneously

Popular Web Servers

- Common web servers include:
  - Apache
  - Microsoft Internet Information Server
- Web servers manage:
  - Client requests
  - Web page delivery

Uniform Resource Locator (URL)

- Every web page requires:
  - Unique identifier
- Web page identified using:
  - Protocol
  - Host
  - Port
  - Path
- URL stands for:
  - Uniform Resource Locator
- URL combines:
  - All identifiers into one format

Components of a URL

- Four identifiers define a web page:
  - Protocol
  - Host
  - Port
  - Path
- Protocol identifies:
  - Client-server application
- Remaining identifiers define:
  - Destination web page

Protocol Identifier

- Protocol specifies:
  - Method used to access web page
- Common protocols:
  - HTTP
  - FTP
- HTTP stands for:
  - HyperText Transfer Protocol
- FTP stands for:
  - File Transfer Protocol

Host Identifier

- Host identifies:
  - Web server
- Host can be:
  - IP address
  - Domain name
- Example IP address:
  - 64.23.56.17
- Example domain name:
  - forouzan.com
- Domain names uniquely identify:
  - Internet hosts

Port Identifier

- Port is:
  - 16-bit integer
- Port identifies:
  - Specific application service
- Most applications use:
  - Well-known port numbers
- Example:
  - HTTP uses port 80
- Port number may be:
  - Explicitly specified if different

Path Identifier

- Path identifies:
  - File location
  - File name
- Path format depends on:
  - Operating system
- UNIX path example:
  - /top/next/last/myfile
- Path lists:
  - Directories from top to bottom
  - File name at end

General URL Formats

```
protocol://host/path
```


```
protocol://host:port/path
```

- First format:
  - Used when default port is assumed
- Second format:
  - Used when custom port is required

Example 2.3: URL Example

`http://www.mhhe.com/compsci/forouzan/`

- Protocol:
  - HTTP
- Host:
  - www.mhhe.com
- Path:
  - compsci/forouzan/
- Path refers to:
  - Forouzan's web page
  - Under directory compsci

Web Documents

- Documents in WWW classified into:
  - Static Documents
  - Dynamic Documents
  - Active Documents
- Classification based on:
  - Creation method
  - Execution location
  - Content behavior

Static Documents

- Static documents are:
  - Fixed-content documents
- Created and stored:
  - At the server
- Client receives:
  - Copy of the document
- Content determined:
  - During file creation
  - Not during use
- User cannot modify:
  - Server contents

Languages for Static Documents

- Static documents prepared using:
  - HTML
  - XML
  - XSL
  - XHTML
- HTML:
  - HyperText Markup Language
- XML:
  - Extensible Markup Language
- XSL:
  - Extensible Style Language
- XHTML:
  - Extensible Hypertext Markup Language

Dynamic Documents

- Dynamic document created:
  - Whenever browser sends request
- Web server executes:
  - Application program
  - Script
- Server sends:
  - Result generated by program
- Fresh document generated for:
  - Each request
- Contents may vary:
  - From one request to another

Example of Dynamic Documents

- Example:
  - Date and time retrieval
- Time and date are:
  - Continuously changing information
- Client requests:
  - Execution of server-side program
- Example program:
  - UNIX date program
- Server sends:
  - Current output of program

Technologies for Dynamic Documents

- Earlier technology:
  - CGI
- CGI stands for:
  - Common Gateway Interface
- Modern scripting technologies:
  - JSP
  - ASP
  - ColdFusion
- JSP uses:
  - Java scripting
- ASP uses:
  - Visual Basic scripting
- ColdFusion embeds:
  - SQL queries inside HTML

Active Documents

- Active documents require:
  - Program execution at client side
- Used for:
  - Animated graphics
  - User interaction
- Browser requests:
  - Active document or script
- Server sends:
  - Executable program
  - Script
- Execution occurs:
  - At client browser

Technologies for Active Documents

- Active documents can use:
  - Java Applets
  - JavaScripts
- Java applets:
  - Written in Java
  - Compiled at server
  - Stored in bytecode format
- JavaScript:
  - Downloaded to client
  - Executed at client site

Comparison of Web Documents

Feature	**Static**	**Dynamic**	**Active**
Created At	Server	Server	Client/Server
Execution	None	Server Side	Client Side
Content Change	Fixed	Variable	Interactive
Examples	HTML Page	Date/Time	Animation
Technologies	HTML/XML	JSP/ASP	JavaScript

Computer Networks

Overview of the Internet
Protocol layering (Book 1 Ch 1)

Application Layer

- Application-Layer Paradigms
- Client-server applications
  - World Wide Web and HTTP
  - FTP
  - Electronic Mail
  - DNS
- Peer-to-peer paradigm
  - P2P Networks
- Case study: BitTorrent (Book 1 Ch 2)

Application Layer

1. **Introduction**
   - Providing Services
   - Application-Layer Paradigms
2. **Standard Client-Server Applications**
   - World Wide Web and HTTP
   - FTP
   - Electronic Mail
   - Domain Name System (DNS)
3. **Peer-to-Peer Paradigm**
   - P2P Networks

HyperText Transfer Protocol (HTTP)

- HTTP stands for:
  - HyperText Transfer Protocol
- HTTP defines:
  - Communication between web client and web server
- Used for:
  - Retrieving web pages from the Web
- HTTP client:
  - Sends request
- HTTP server:
  - Returns response

HTTP Communication

- Server uses:
  - Port number 80
- Client uses:
  - Temporary port number
- HTTP uses services of:
  - TCP
- TCP provides:
  - Connection-oriented communication
  - Reliable communication

Role of TCP in HTTP

- Before HTTP transaction:
  - TCP connection established
- After transaction:
  - TCP connection terminated
- TCP handles:
  - Error control
  - Message loss recovery
- Client and server do not need to:
  - Manage transmission errors

HTTP and Multiple Objects

- Web pages may contain:
  - Multiple linked objects
- Objects may be stored:
  - On same server
  - On different servers
- Different servers require:
  - Separate TCP connections
- Same server offers:
  - Two connection strategies

Nonpersistent vs Persistent Connections

- Two HTTP connection strategies:
  - Nonpersistent connections
  - Persistent connections
- Nonpersistent:
  - One TCP connection per request/response
- Persistent:
  - One TCP connection for multiple objects
- HTTP versions:
  - Before HTTP 1.1 $\rightarrow$ Nonpersistent
  - HTTP 1.1 $\rightarrow$ Persistent by default

Nonpersistent Connections

- One TCP connection used for:
  - One request
  - One response
- Connection closed after:
  - Response transmission
- New request requires:
  - New TCP connection
- Suitable for:
  - Simple communication

Steps in Nonpersistent Connections

1. Client opens TCP connection and sends request
2. Server sends response and closes connection
3. Client reads data until end-of-file marker
4. Client closes connection

- Each request-response pair:
  - Uses separate connection

Overhead in Nonpersistent Connections

- Suppose a file contains:
  - N linked images
- Total TCP connections required:
  - N + 1
- Reason:
  - One connection for main file
  - N connections for images
- Nonpersistent strategy causes:
  - High server overhead
- Server requires:
  - Separate buffer for each connection

Comparison of HTTP Connection Types

Feature	Nonpersistent	Persistent
TCP Connections	Multiple	Single
Connection Reuse	No	Yes
Overhead	High	Lower
Efficiency	Lower	Higher
HTTP Version	Before 1.1	HTTP 1.1 Default

Example 2.4: Nonpersistent Connection

- Client accesses:
  - Text file containing one image link
- Text file and image:
  - Stored on same server
- Nonpersistent HTTP requires:
  - Separate TCP connection for each object
- Total connections required:
  - Two connections

Connection Establishment in HTTP

- Each TCP connection requires:
  - Three-way handshake
- At least three messages needed:
  - To establish connection
- Request can be sent:
  - Along with third handshake message
- After connection establishment:
  - Object transferred

Connection Termination

- After object transfer:
  - TCP connection terminated
- TCP termination requires:
  - Three handshake messages
- Each object retrieval causes:
  - One connection establishment
  - One connection termination

Total Operations in Example 2.4

- Two objects retrieved:
  - Main text file
  - Image file
- Total operations:
  - Two TCP connection establishments
  - Two TCP connection terminations
- Each connection introduces:
  - Additional delay
  - Additional overhead

Overhead of Nonpersistent Connections

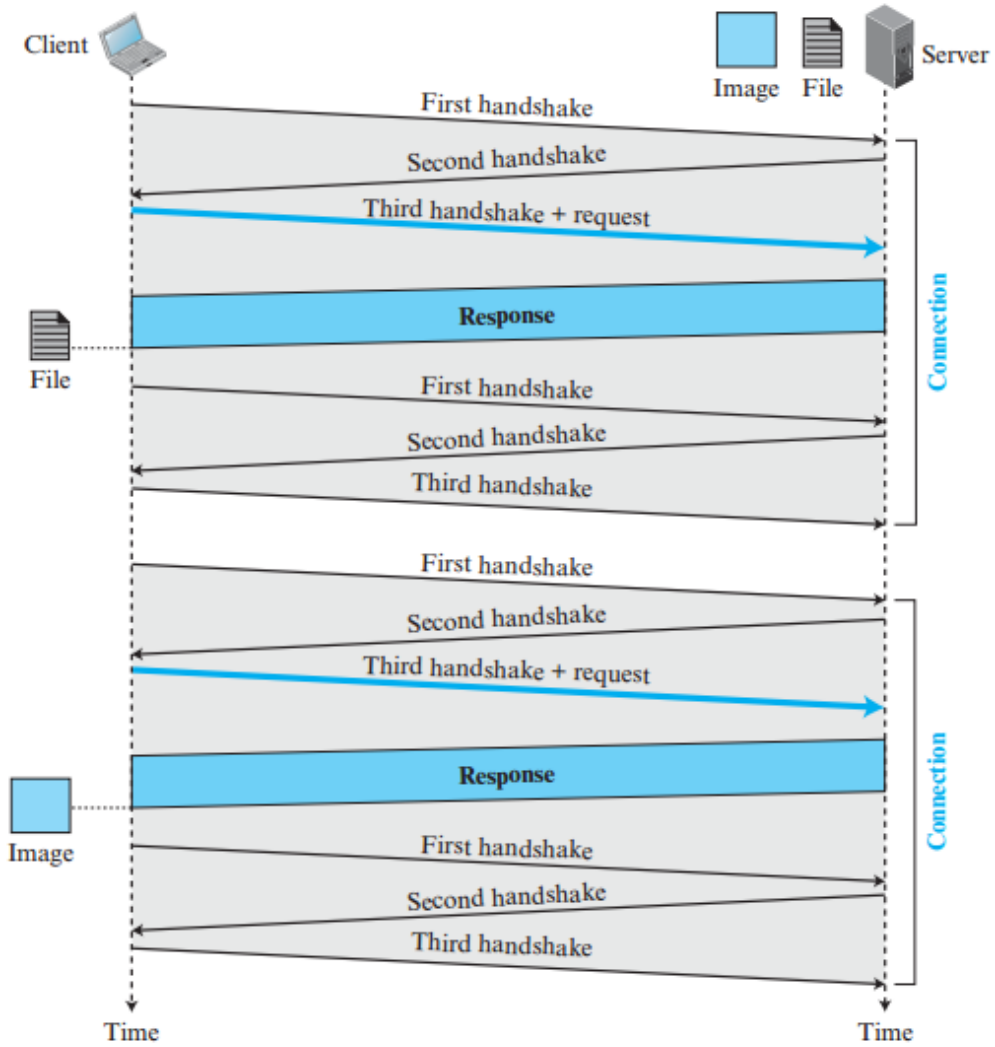
- Retrieving many objects increases:
  - Round-trip times
- Example:
  - 10 or 20 objects require many handshakes
- Handshake overhead becomes:
  - Significant
- Nonpersistent connections reduce:
  - Communication efficiency

Resource Usage in Nonpersistent Connections

- Each TCP connection requires:
  - Buffers
  - Variables
  - System resources
- Resource allocation needed at:
  - Client
  - Server
- Greater burden placed on:
  - Server
- Multiple connections increase:
  - Server workload

Example 2.4

Figure 2.10 *Example 2.4*



Example 2.4 Illustration

Connection 1
Client $\Longleftrightarrow$ Server
Retrieve Text File
Connection 2

Client $\Longleftrightarrow$ Server
Retrieve Image File

- Separate TCP connection for each object
- Each connection requires setup and termination

Persistent Connections

- HTTP version 1.1 uses:
  - Persistent connections by default
- In persistent connection:
  - TCP connection remains open
- Same connection used for:
  - Multiple requests
  - Multiple responses
- Eliminates need for:
  - Repeated connection establishment

Operation of Persistent Connections

- Server keeps connection open:
  - After sending response
- Connection closed when:
  - Client requests closure
  - Time-out occurs
- Multiple objects can be retrieved:
  - Through single TCP connection

Data Length Information

- Sender usually includes:
  - Length of transmitted data
- Data length helps client:
  - Detect end of response
- Some situations prevent:
  - Knowing data length in advance

Unknown Data Length Situations

- Unknown data length occurs with:
  - Dynamic documents
  - Active documents
- Server informs client:
  - Length is unknown
- Server closes connection:
  - After sending complete data
- Connection closure indicates:
  - End of data transmission

Advantages of Persistent Connections

- Persistent connections save:
  - Time
  - Resources
- Only one set of:
  - Buffers
  - Variables
- Resource allocation needed:
  - Only once per connection
- Reduces:
  - Server overhead
  - Client overhead

Reduction in Round Trip Time

- Persistent connections avoid:
  - Repeated TCP handshakes
- Saves round trip time for:
  - Connection establishment
  - Connection termination
- Improves:
  - Communication efficiency
  - Web performance

Persistent Connection Illustration

Single TCP Connection

Client $\Longleftrightarrow$ Server

Request 1 $\rightarrow$ Response 1

Request 2 $\rightarrow$ Response 2

Request 3 $\rightarrow$ Response 3

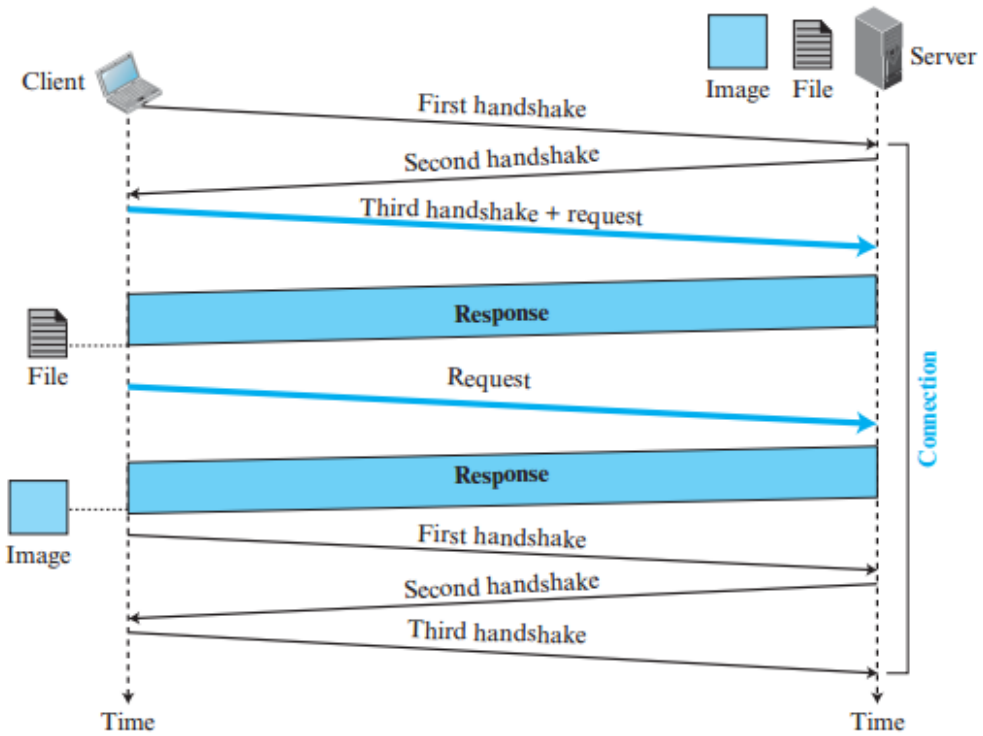
- One connection supports multiple transactions

Nonpersistent vs Persistent Connections

Feature	Nonpersistent	Persistent
TCP Connections	Multiple	Single
Setup Overhead	High	Low
Resource Usage	High	Lower
Round Trip Delay	More	Less
HTTP Version	Before 1.1	HTTP 1.1 Default

Example 2.5

Figure 2.11 *Example 2.5*



Example 2.5: Persistent Connection

- Same scenario as Example 2.4
- Client accesses:
  - Text file
  - Image file
- Both files located on:
  - Same server
- Persistent HTTP connection used:
  - Single TCP connection

Connection Handling in Persistent HTTP

- Only one:
  - Connection establishment
  - Connection termination
- Same TCP connection reused for:
  - Multiple object transfers
- Eliminates repeated:
  - TCP handshakes

Separate Requests in Persistent Connection

- Main text file retrieved:
  - Through first request
- Image file retrieved:
  - Through separate request
- Both requests use:
  - Same TCP connection
- Persistent connection keeps:
  - Communication channel open

Advantages in Example 2.5

- Reduced:
  - Connection setup overhead
  - Connection termination overhead
- Saves:
  - Round-trip time
  - System resources
- Requires:
  - Only one set of buffers
  - Only one set of variables
- Improves:
  - Communication efficiency

Example 2.5 Illustration

Single TCP Connection
Client $\Longleftrightarrow$ Server

Request 1 $\rightarrow$ Retrieve Text File

Request 2 $\rightarrow$ Retrieve Image File
Single Connection Termination

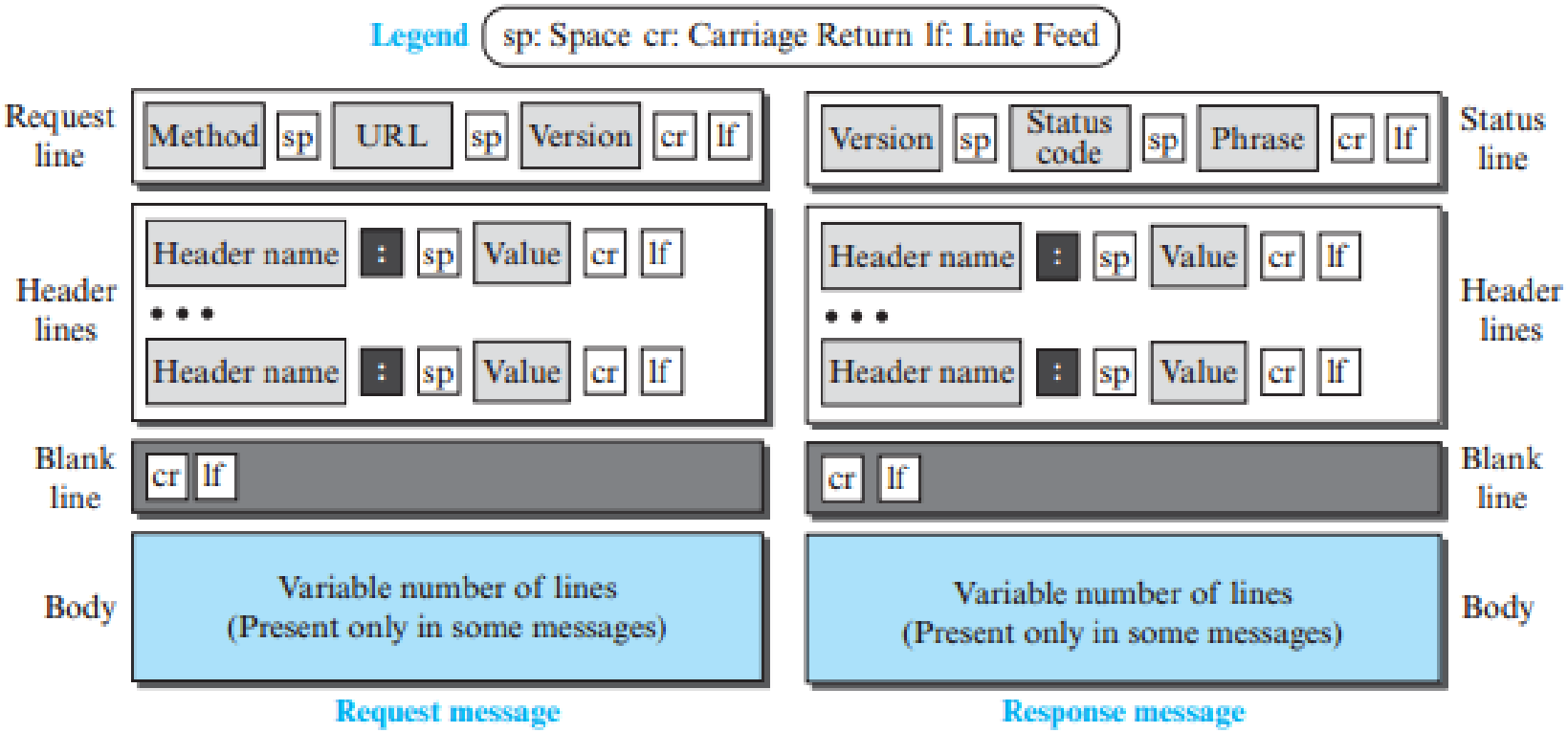
- Multiple requests handled using one TCP connection

Example 2.4 vs Example 2.5

Feature	**Example 2.4**	**Example 2.5**
Connection Type	Nonpersistent	Persistent
TCP Connections	Two	One
Connection Setup	Multiple	Single
Connection Termination	Multiple	Single
Efficiency	Lower	Higher
Overhead	Higher	Lower

Formats of the request and response messages

Figure 2.12 *Formats of the request and response messages*



HTTP Message Formats

- HTTP defines formats for:
  - Request messages
  - Response messages
- Request and response formats:
  - Similar in structure
- Each message contains:
  - Four sections
- Message formats standardized by:
  - HTTP protocol

HTTP Request Message Structure

- First section called:
  - Request Line
- Other sections include:
  - Header section
  - Blank line
  - Body section
- Request message sent by:
  - HTTP client

HTTP Response Message Structure

- First section called:
  - Status Line
- Other sections include:
  - Header section
  - Blank line
  - Body section
- Response message sent by:
  - HTTP server

Comparison of Request and Response Messages

- Request and response messages have:
  - Same section names
- However:
  - Contents of sections may differ
- Request message contains:
  - Client request information
- Response message contains:
  - Server response information

HTTP Message Format Illustration

Request Message	**Response Message**
Request Line	Status Line
Header Section	Header Section
Blank Line	Blank Line
Message Body	Message Body

- Structures are similar
- Contents differ according to message type

Methods

Table 2.1 *Methods*

Method	*Action*
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

Request Header Names

Table 2.2 *Request Header Names*

Header	*Description*
User-agent	Identifies the client program
Accept	Shows the media format the client can accept
Accept-charset	Shows the character set the client can handle
Accept-encoding	Shows the encoding scheme the client can handle
Accept-language	Shows the language the client can accept
Authorization	Shows what permissions the client has
Host	Shows the host and port number of the client
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Cookie	Returns the cookie to the server (explained later)
If-Modified-Since	If the file is modified since a specific date

HTTP Request Message

- First line of request message:
  - Request Line
- Request line contains:
  - Method
  - URL
  - Version
- Fields separated by:
  - Single space
- Line terminated by:
  - Carriage Return (CR)
  - Line Feed (LF)

Structure of Request Line

```
Method    URL    Version
```


- Method:
  - Type of request
- URL:
  - Address of web page
- Version:
  - HTTP protocol version

HTTP Methods

- HTTP version 1.1 defines:
  - Several request methods
- Common methods include:
  - GET
  - HEAD
  - PUT
  - POST
  - TRACE
  - DELETE
  - CONNECT
  - OPTIONS

GET and HEAD Methods

- GET method:
  - Most commonly used
  - Retrieves web page
- GET request usually contains:
  - Empty body
- HEAD method:
  - Retrieves only header information
- Used for:
  - Checking modification time
  - Testing URL validity
- HEAD response contains:
  - Header only
  - Empty body

PUT and POST Methods

- PUT method:
  - Opposite of GET
  - Uploads new web page to server
- PUT allowed only:
  - With permission
- POST method:
  - Sends information to server
- POST used for:
  - Adding information
  - Modifying web pages

Other HTTP Methods

- TRACE method:
  - Used for debugging
  - Server echoes request back
- DELETE method:
  - Deletes web page from server
- CONNECT method:
  - Reserved for proxy server use
- OPTIONS method:
  - Retrieves properties of web page

URL and Version Fields

- URL field defines:
  - Address and name of web page
- Version field specifies:
  - HTTP protocol version
- Current HTTP version:
  - HTTP/1.1

Request Header Lines

- Request message may contain:
  - Zero or more header lines
- Header lines provide:
  - Additional client information
- Example:
  - Requesting special document format
- Header line format:
  - Header Name : Value

Format of Header Lines

```
Header-Name :  Header-Value
```

- Header name:
  - Identifies information type
- Header value:
  - Associated parameter value

Body of Request Message

- Request body may be:
  - Present or absent
- Body commonly used with:
  - PUT method
  - POST method
- Body may contain:
  - Comments
  - Uploaded files
  - Data for web page modification

Summary of HTTP Methods

Method	**Purpose**
GET	Retrieve web page
HEAD	Retrieve header only
PUT	Upload new page
POST	Send/modify data
TRACE	Debugging
DELETE	Delete web page
CONNECT	Proxy communication
OPTIONS	Request page properties

Response Header Names

Table 2.3 *Response Header Names*

Header	*Description*
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Server	Gives information about the server
Set-Cookie	The server asks the client to save a cookie
Content-Encoding	Specifies the encoding scheme
Content-Language	Specifies the language
Content-Length	Shows the length of the document
Content-Type	Specifies the media type
Location	To ask the client to send the request to another site
Accept-Ranges	The server will accept the requested byte-ranges
Last-modified	Gives the date and time of the last change

HTTP Response Message

- Response message consists of:
  - Status line
  - Header lines
  - Blank line
  - Body
- Response message sent by:
  - HTTP server
- Body may be:
  - Present or absent

Status Line in Response Message

- First line called:
  - Status Line
- Status line contains:
  - Version
  - Status Code
  - Status Phrase
- Fields separated by:
  - Spaces
- Line terminated by:
  - Carriage Return (CR)
  - Line Feed (LF)

Structure of Status Line

```
Version   Status-Code   Status-Phrase
```

- Version:
  - HTTP protocol version
- Status Code:
  - Numeric request status
- Status Phrase:
  - Text explanation of status

HTTP Version Field

- First field specifies:
  - HTTP protocol version
- Current version:
  - HTTP/1.1
- Helps client identify:
  - Supported protocol features

HTTP Status Codes

- Status code contains:
  - Three digits
- Indicates:
  - Result of client request
- Status codes grouped into:
  - 100 range
  - 200 range
  - 300 range
  - 400 range
  - 500 range

Meaning of Status Code Ranges

- 100 range:
  - Informational messages
- 200 range:
  - Successful request
- 300 range:
  - Redirection to another URL
- 400 range:
  - Client-side errors
- 500 range:
  - Server-side errors

Status Phrase

- Status phrase:
  - Text explanation of status code
- Helps user understand:
  - Meaning of response
- Example:
  - 200 OK
  - 404 Not Found

Response Header Lines

- Response may contain:
  - Zero or more header lines
- Header lines provide:
  - Additional information
- Information sent from:
  - Server to client
- Example:
  - Information about document

Format of Response Header Lines

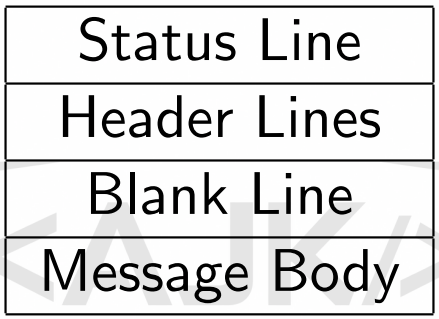
```
Header-Name : Header-Value
```


- Header name:
  - Defines information type
- Header value:
  - Defines associated value

Body of Response Message

- Body contains:
  - Requested document
- Document transferred from:
  - Server to client
- Body absent when:
  - Response is an error message
- Browser uses body to:
  - Display web content

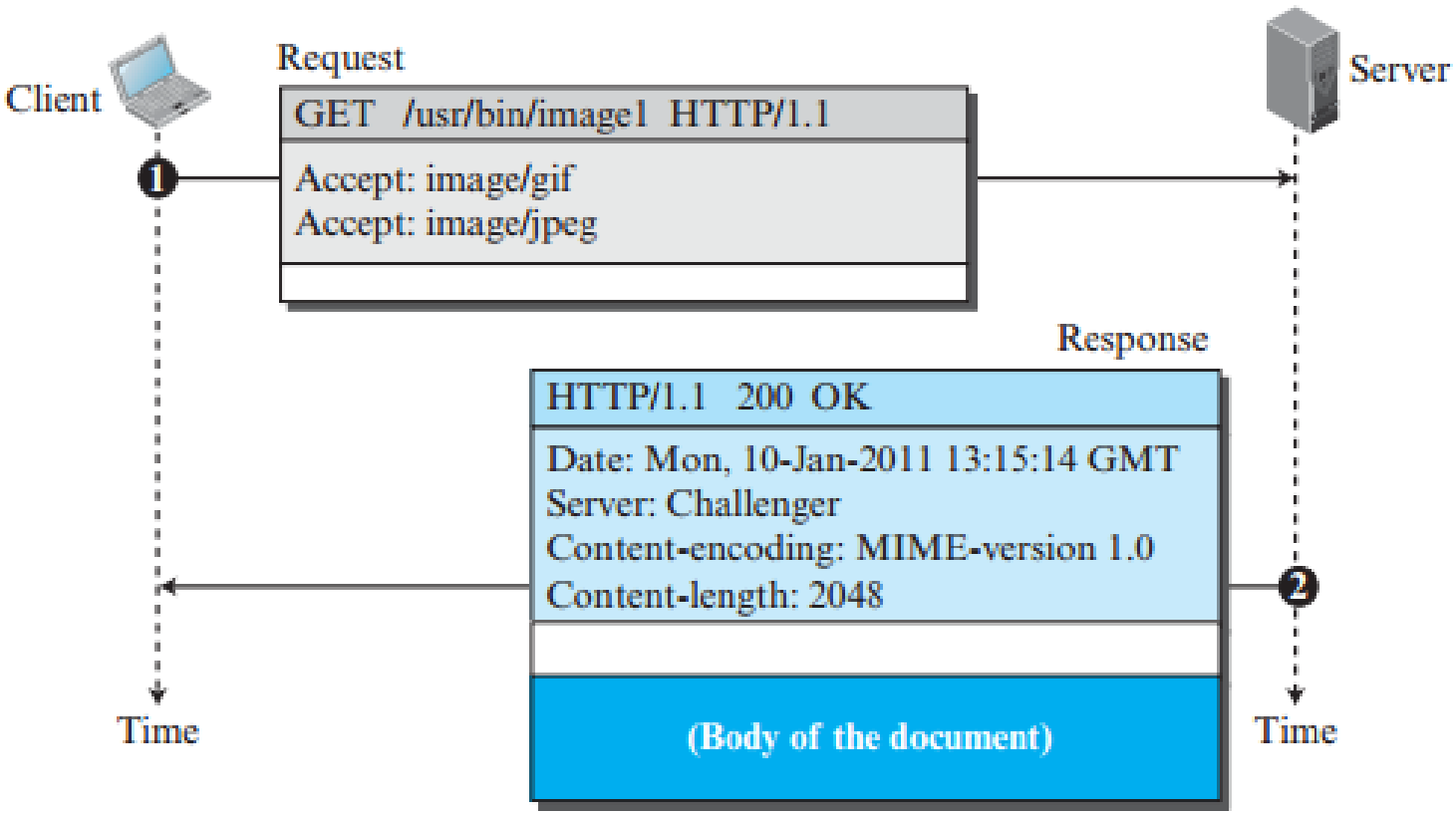
HTTP Response Message Structure



- Structure used in all HTTP responses

Example 2.6

Figure 2.13 *Example 2.6*



Example 2.6: Retrieving a Document

- Example demonstrates:
  - HTTP request and response
- Client retrieves:
  - Image document
- HTTP method used:
  - GET
- Requested path:
  - /usr/bin/image1

HTTP Request Line in Example 2.6

- Request line contains:
  - Method
  - URL
  - HTTP version
- Method:
  - GET
- URL:
  - /usr/bin/image1
- Version:
  - HTTP/1.1

Request Header Information

- Request header contains:
  - Two header lines
- Header lines indicate:
  - Acceptable image formats
- Supported formats:
  - GIF
  - JPEG
- Request body:
  - Not present

HTTP Response Message in Example 2.6

- Response message contains:
  - Status line
  - Four header lines
  - Body
- Server sends:
  - Requested image document

Response Header Information

- Response headers provide:
  - Date information
  - Server information
  - Content encoding
  - Document length
- Content encoding specifies:
  - MIME version
- Document length defines:
  - Size of transmitted data

Body of the Response

- Body follows:
  - Response headers
- Body contains:
  - Requested image document
- Browser uses body to:
  - Display retrieved content

Illustration of Example 2.6

Request Message
GET /usr/bin/image1 HTTP/1.1 Accept: GIF Accept: JPEG
No Body

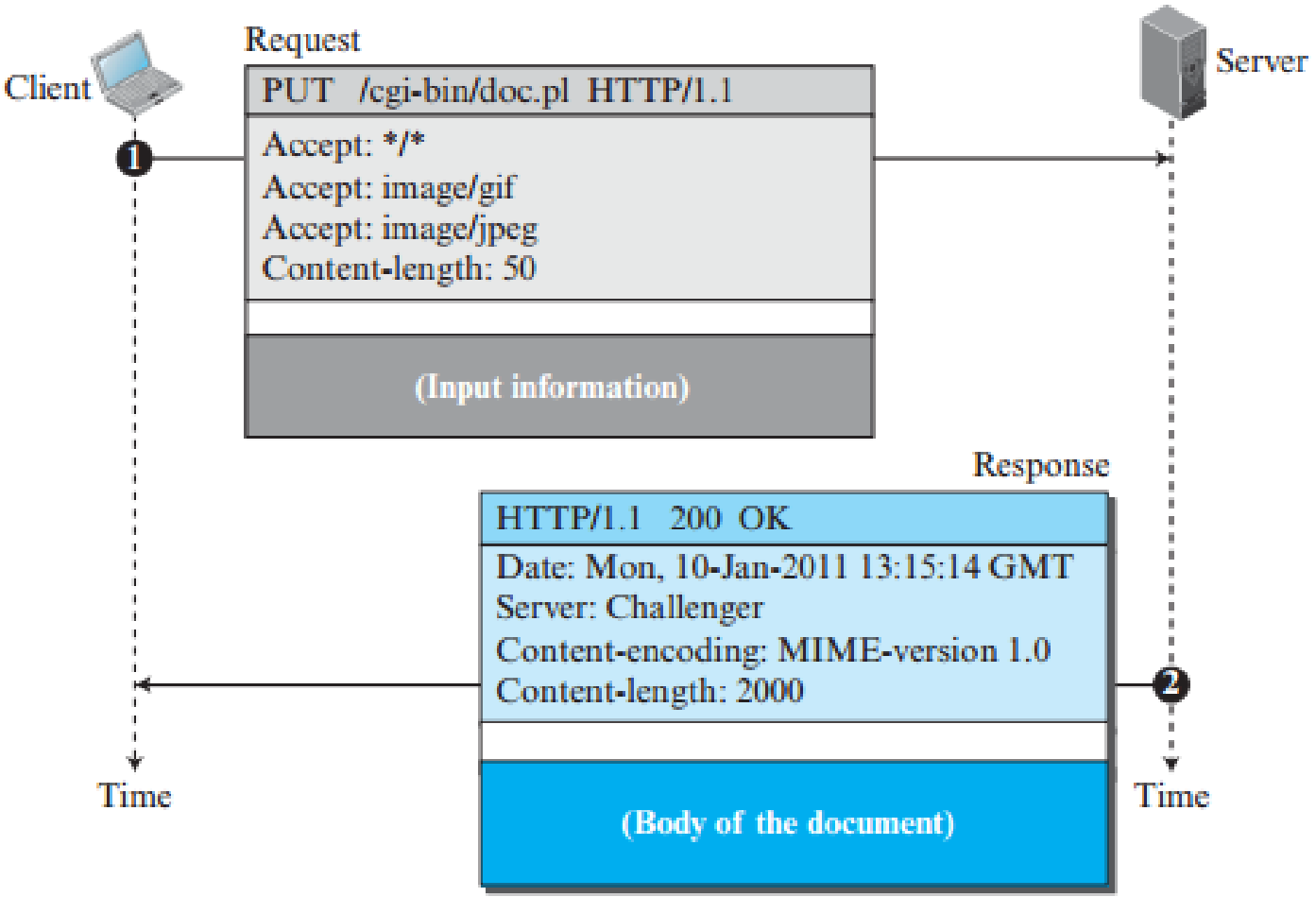
Response Message
HTTP/1.1 200 OK Date Information Server Information MIME Encoding Content Length
Image Document Body

Key Points from Example 2.6

- GET method used to:
  - Retrieve resource
- Request specifies:
  - Accepted document formats
- Response provides:
  - Status information
  - Document details
  - Requested content
- MIME encoding helps:
  - Identify document format

Example 2.7

Figure 2.14 *Example 2.7*



Example 2.7: Uploading a Web Page

- Client wants to:
  - Send a web page to server
- HTTP method used:
  - PUT
- Purpose of PUT:
  - Post new web page on server

Request Line in Example 2.7

- Request line contains:
  - Method
  - URL
  - HTTP version
- Method:
  - PUT
- Version:
  - HTTP/1.1
- URL specifies:
  - Destination web page location

Request Headers and Body

- Request message contains:
  - Four header lines
- Headers provide:
  - Additional request information
- Request body contains:
  - Web page to be uploaded
- PUT request transfers:
  - Entire document to server

Response Message in Example 2.7

- Server sends:
  - Response message
- Response contains:
  - Status line
  - Four header lines
  - Body
- Status line indicates:
  - Result of upload operation

CGI Document in Response

- Response body contains:
  - Created CGI document
- CGI stands for:
  - Common Gateway Interface
- CGI document generated by:
  - Server-side processing
- Response confirms:
  - Successful document creation

Illustration of Example 2.7

Request Message
PUT URL HTTP/1.1
Header Line 1
Header Line 2
Header Line 3
Header Line 4
Web Page Body

Response Message
HTTP/1.1 Status Code
Header Line 1
Header Line 2
Header Line 3
Header Line 4
CGI Document Body

Key Points from Example 2.7

- PUT method used for:
  - Uploading documents
- Request body carries:
  - Actual web page content
- Server processes uploaded page:
  - Creates CGI response
- Response confirms:
  - Status of upload operation

Conditional Request

- HTTP client can add:
  - Conditions in request message
- Server checks:
  - Whether condition is satisfied
- If condition is met:
  - Requested web page sent
- Otherwise:
  - Server informs client

Purpose of Conditional Requests

- Conditional requests help:
  - Avoid unnecessary data transfer
- Client requests document only when:
  - Specific condition holds true
- Improves:
  - Network efficiency
  - Communication performance

If-Modified-Since Header

- Common conditional header:
  - If-Modified-Since
- Client specifies:
  - Date and time
- Server sends page only if:
  - Page modified after specified time
- Used to check:
  - Updated version of document

Working of If-Modified-Since

- Client sends request with:
  - If-Modified-Since header
- Server compares:
  - Modification time of web page
- If page updated:
  - Server sends latest page
- If page unchanged:
  - Server avoids sending duplicate data

Example of Conditional Request

```
If-Modified-Since:  10 Jan 2026 10:00:00
```

- Server sends page only if:
  - Modified after specified date and time
- Otherwise:
  - No updated page transferred

Advantages of Conditional Requests

- Reduces:
  - Network traffic
  - Unnecessary retransmission
- Saves:
  - Bandwidth
  - Time
- Improves:
  - Web efficiency
  - Response speed
- Useful for:
  - Frequently accessed web pages

Example 2.8: Conditional Request

- Client sends:
  - Conditional HTTP request
- Condition based on:
  - Modification date and time
- Request uses:
  - If-Modified-Since header
- Goal:
  - Retrieve page only if updated

HTTP Request in Example 2.8

```
GET http://www.commonServer.com/information/file1 HTTP/1.1
If-Modified-Since: Thu, Sept 04 00:00:00 GMT
```

- GET method used to:
  - Retrieve web page
- Header specifies:
  - Required modification condition

Meaning of the Condition

- Client requests page only if:
  - Modified after Sept 04
- Server checks:
  - Last modification time of file
- If file unchanged:
  - Server avoids sending document

HTTP Response in Example 2.8

HTTP/1.1 304 Not Modified Date: Sat, Sept 06 08:16:22 GMT Server: commonServer.com
(Empty Body)

Status Code 304 Not Modified

- Response status code:
  - 304 Not Modified
- Indicates:
  - Requested file not updated
- Server determines:
  - File unchanged after specified date
- No document retransmission required

Response Headers in Example 2.8

- Response contains:
  - Date header
  - Server header
- Date header specifies:
  - Time of response generation
- Server header specifies:
  - Name of responding server

Empty Response Body

- Response body is:
  - Empty
- Reason:
  - File not modified
- Sending document again would:
  - Waste bandwidth
- Client can continue using:
  - Cached copy of file

Advantages of Example 2.8

- Reduces:
  - Network traffic
  - Duplicate transmissions
- Saves:
  - Bandwidth
  - Response time
- Improves:
  - Web efficiency
  - Cache utilization

Computer Networks

Overview of the Internet
Protocol layering (Book 1 Ch 1)

Application Layer

- Application-Layer Paradigms
- Client-server applications
  - World Wide Web and HTTP
  - FTP
  - Electronic Mail
  - DNS
- Peer-to-peer paradigm
  - P2P Networks
- Case study: BitTorrent (Book 1 Ch 2)

Application Layer

1. **Introduction**
   - Providing Services
   - Application-Layer Paradigms
2. **Standard Client-Server Applications**
   - World Wide Web and HTTP
   - FTP
   - Electronic Mail
   - Domain Name System (DNS)
3. **Peer-to-Peer Paradigm**
   - P2P Networks

Cookies

- Original Web designed as:
  - Stateless entity
- Stateless communication:
  - Client sends request
  - Server sends response
  - Connection information forgotten
- Original Web purpose:
  - Retrieve public documents
- Modern Web applications require:
  - Client information retention

Need for Cookies

- Cookies introduced to:
  - Remember client information
- Required by applications such as:
  - Electronic shopping
  - Restricted-access websites
  - Web portals
  - Advertising agencies
- Cookies provide:
  - State management in Web communication

Electronic Shopping and Cookies

- E-commerce websites allow users to:
  - Browse products
  - Select items
  - Store items in shopping cart
  - Pay online
- Cookies store:
  - Shopping cart information
- Cookie updated whenever:
  - New item selected

Restricted Access and Portals

- Some websites allow access only to:
  - Registered users
- Cookies identify:
  - Authorized clients
- Web portals use cookies to:
  - Store user preferences
  - Remember favorite pages
- Returning users receive:
  - Customized web pages

Advertising and Cookies

- Advertising agencies use cookies to:
  - Track user behavior
- Banner advertisements may contain:
  - URL of advertising agency
- Clicking advertisement sends:
  - Request to advertising server
- Cookies help agencies:
  - Build user interest profiles

Creating and Storing Cookies

1. Server receives client request
2. Server creates cookie containing client information
3. Cookie included in server response
4. Browser stores cookie in cookie directory

- Cookie directory organized by:
  - Server domain name

Contents of a Cookie

- Cookie may contain:
  - Client domain name
  - User information
  - Registration number
  - Timestamp
  - Other implementation details
- Cookie created by:
  - Server

Using Cookies

- Browser checks:
  - Cookie directory before sending request
- If matching cookie found:
  - Cookie attached to request
- Server recognizes:
  - Returning client
- Cookie contents:
  - Not disclosed to user

Cookies in E-Commerce

- Cookie stores:
  - Item number
  - Unit price
- Cookie updated after:
  - Each item selection
- During checkout:
  - Cookie retrieved
  - Total charge calculated

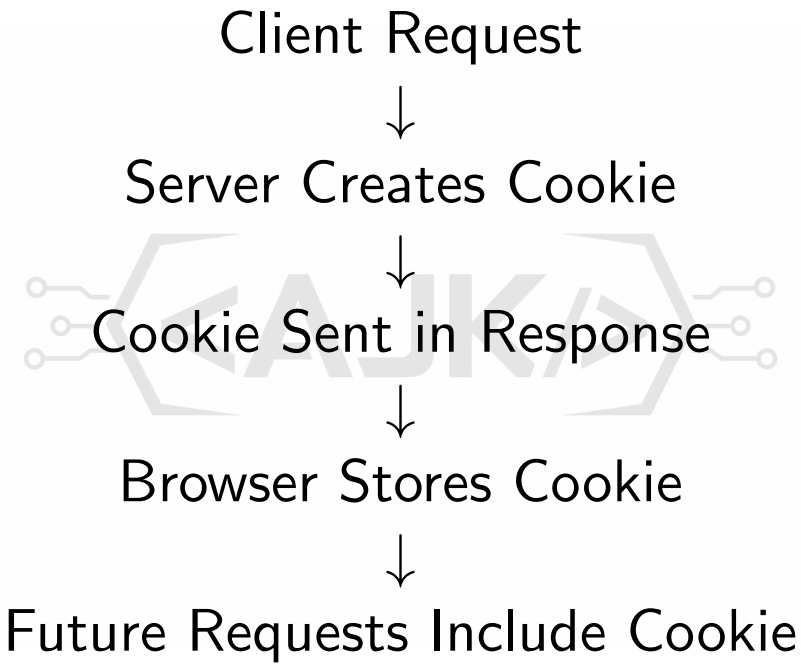
Cookies for Registered Users

- Website sends cookie:
  - During first registration
- Future access allowed only for:
  - Clients sending valid cookie
- Cookie acts as:
  - User identification mechanism

Cookies and Privacy Issues

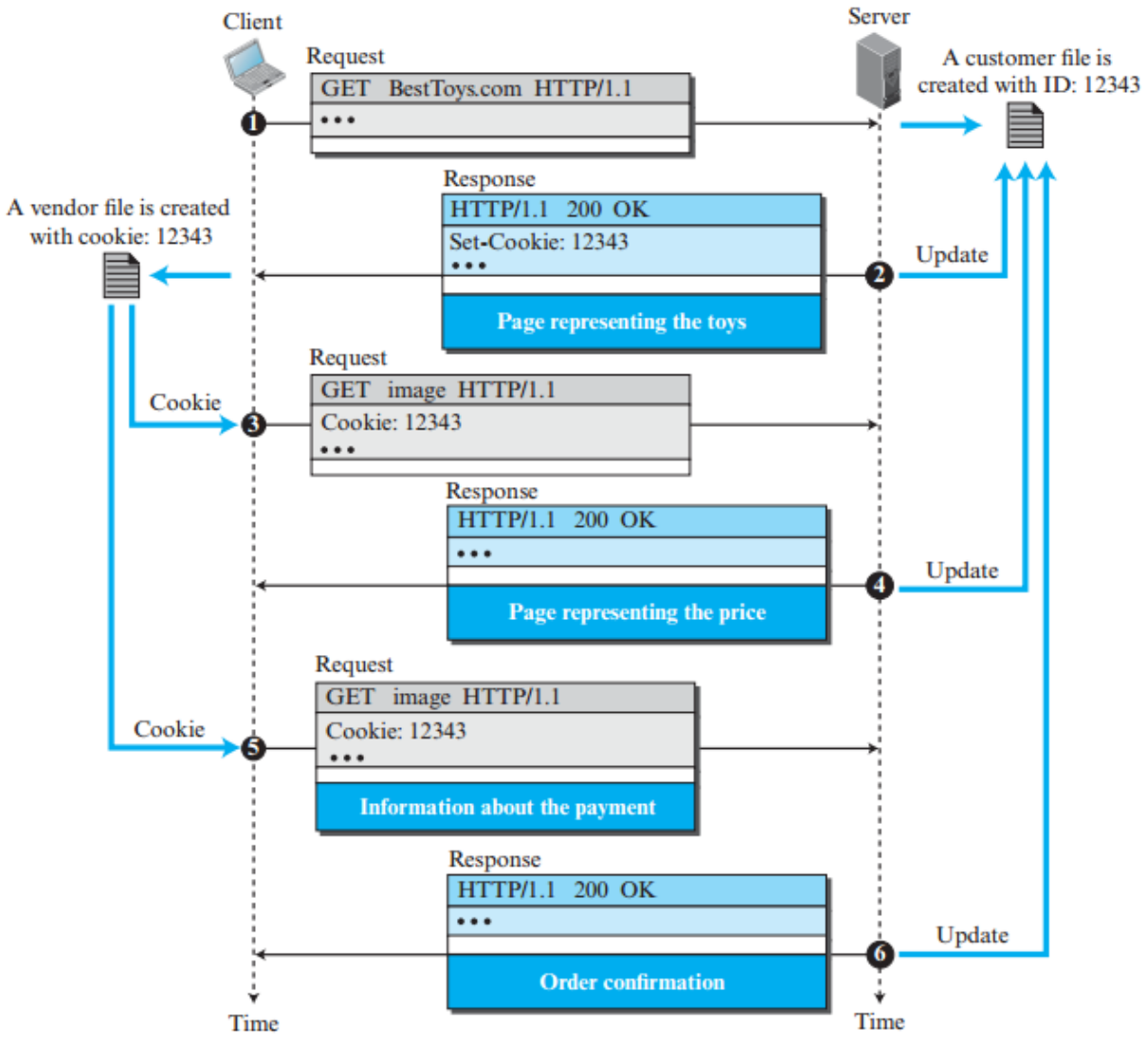
- Advertising agencies may:
  - Track browsing behavior
- User interests stored in:
  - Databases
- Information may be:
  - Sold to third parties
- Cookie tracking creates:
  - Privacy concerns
- Need for:
  - Privacy regulations

Cookie Operation Illustration



Example 2.9

Figure 2.15 *Example 2.9*



Example 2.9: Cookies in Electronic Shopping

- Example demonstrates:
  - Use of cookies in e-commerce
- Electronic store:
  - BestToys
- Shopper accesses:
  - BestToys website
- Cookies help server:
  - Remember customer information

Initial Client Request

- Shopper browser sends:
  - Request to BestToys server
- Server creates:
  - Empty shopping cart
- Shopping cart assigned:
  - Unique ID
- Example cart ID:
  - 12343

Server Response with Cookie

- Server response contains:
  - Images of toys
  - Links for toy selection
- Response also includes:
  - Set-Cookie header
- Cookie value:
  - 12343
- Browser stores cookie:
  - In file named BestToys

Cookie Privacy

- Cookie stored by:
  - Browser
- Cookie not revealed to:
  - Shopper
- Browser automatically handles:
  - Cookie storage
  - Cookie transmission

Selecting a Toy

- Shopper clicks:
  - Selected toy
- Browser sends:
  - New request
- Request contains:
  - Cookie header line
- Cookie value included:
  - 12343

Server Identifies Returning Client

- Server reads:
  - Cookie value 12343
- Server understands:
  - Shopper is returning client
- Server searches for:
  - Shopping cart with ID 12343
- Selected toy added to:
  - Existing shopping cart

Payment Processing

- Server sends response containing:
  - Total price
  - Payment request
- Shopper provides:
  - Credit card information
- Client sends another request:
  - Including cookie value 12343

Order Confirmation

- Server receives:
  - Request with cookie 12343
- Server verifies:
  - Shopping cart information
- Server accepts:
  - Order
  - Payment
- Confirmation sent back:
  - In response message

Future Visits to the Store

- Browser sends cookie again:
  - During future visits
- Server retrieves:
  - Stored customer information
- Store remembers:
  - Shopper details
  - Previous interactions
- Cookies provide:
  - Personalized shopping experience

Cookie Workflow in Example 2.9

Client Request

↓

Server Creates Cart ID = 12343

↓

Set-Cookie Sent to Browser

↓

Browser Stores Cookie

↓

Future Requests Include Cookie 12343

↓

Server Retrieves Shopping Cart

Advantages of Cookies in E-Commerce

- Cookies help:
  - Maintain shopping sessions
  - Track customer selections
  - Store user preferences
- Improves:
  - User convenience
  - Shopping efficiency
- Eliminates need for:
  - Re-entering information repeatedly

Web Caching and Proxy Server

- HTTP supports:
  - Proxy servers
- Proxy server:
  - Computer that stores copies of recent responses
- Purpose of caching:
  - Reduce repeated requests to original server
- Cached responses reused for:
  - Future client requests

Working of a Proxy Server

- Client sends request to:
  - Proxy server
- Proxy server checks:
  - Local cache memory
- If response found:
  - Response sent directly to client
- If response not found:
  - Proxy forwards request to original server
- Incoming response:
  - Stored in proxy cache
  - Sent to client

Advantages of Proxy Servers

- Reduces:
  - Load on original server
- Decreases:
  - Network traffic
- Improves:
  - Latency
  - Response time
- Frequently requested pages:
  - Delivered faster

Client Configuration for Proxy Server

- Client must be configured to:
  - Access proxy server
- Client does not contact:
  - Target server directly
- Proxy server acts as:
  - Intermediate system

Dual Role of Proxy Server

- Proxy server acts as:
  - Server
  - Client
- When cached response exists:
  - Acts as server
  - Sends response to client
- When cached response absent:
  - Acts as client
  - Sends request to target server

Proxy Server Communication Flow

1. Client sends request to proxy server
2. Proxy checks cache
3. If cached:
   - Response sent to client
4. If not cached:
   - Proxy sends request to target server
5. Target server sends response
6. Proxy stores response in cache
7. Proxy forwards response to client

Proxy Server Location

- Proxy servers usually located:
  - Near client side
- Multiple proxy levels possible:
  - Client level
  - Company level
  - ISP level
- Creates:
  - Hierarchy of proxy servers

Client-Level Proxy Server

- Single client computer can act as:
  - Small proxy server
- Stores:
  - Frequently accessed responses
- Benefits:
  - Faster local access

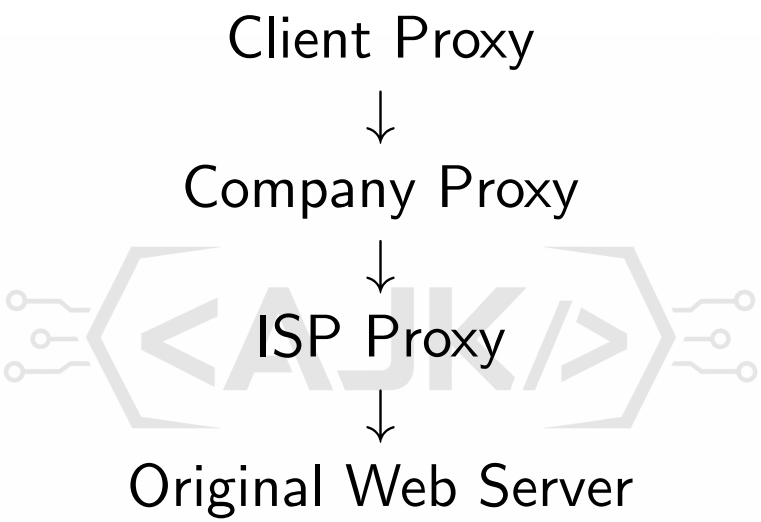
Company-Level Proxy Server

- Proxy installed on:
  - Company LAN
- Purpose:
  - Reduce incoming traffic
  - Reduce outgoing traffic
- Shared cache improves:
  - Organization-wide efficiency

ISP-Level Proxy Server

- ISP may install:
  - Large proxy server
- Used for:
  - Many customers
- Benefits:
  - Reduced ISP traffic
  - Faster content delivery

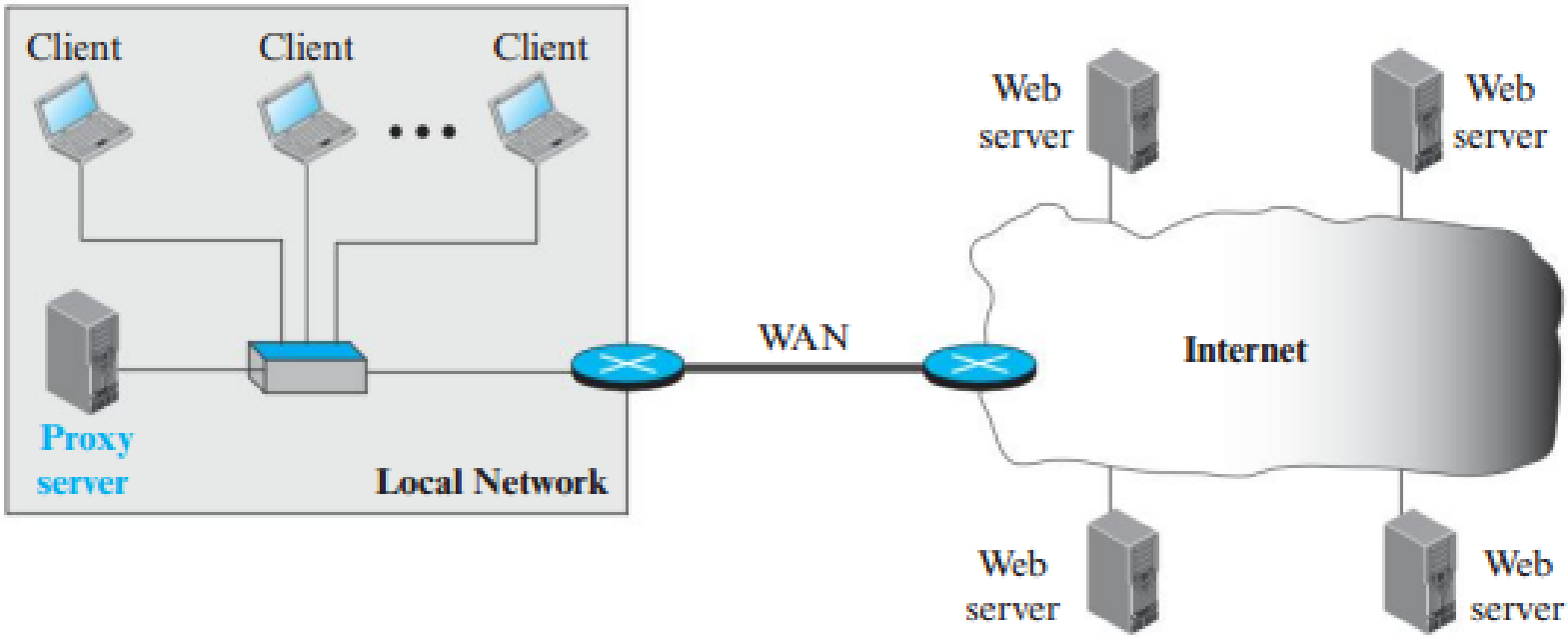
Hierarchy of Proxy Servers



- Multiple proxy levels improve caching efficiency

Example 2.10

Figure 2.16 *Example of a proxy server*



Example 2.10: Proxy Server in Local Network

- Example demonstrates:
  - Use of proxy server in local network
- Local network examples:
  - Campus network
  - Company network
- Proxy server installed:
  - Inside local network

Client Request Handling

- HTTP request created by:
  - Client browser
- Request first directed to:
  - Proxy server
- Client does not contact:
  - Web server directly
- Proxy acts as:
  - Intermediate system

When Web Page Exists in Cache

- Proxy server checks:
  - Cache memory
- If requested web page exists:
  - Proxy sends response directly
- No need to contact:
  - Internet web server
- Result:
  - Faster response
  - Reduced traffic

When Web Page Does Not Exist in Cache

- If page not found in cache:
  - Proxy acts as client
- Proxy sends request to:
  - Web server on Internet
- Web server processes:
  - HTTP request
- Response returned to:
  - Proxy server

Caching the Response

- Proxy server receives:
  - Response from Internet server
- Proxy creates:
  - Copy of response
- Copy stored in:
  - Proxy cache
- Response then forwarded to:
  - Requesting client

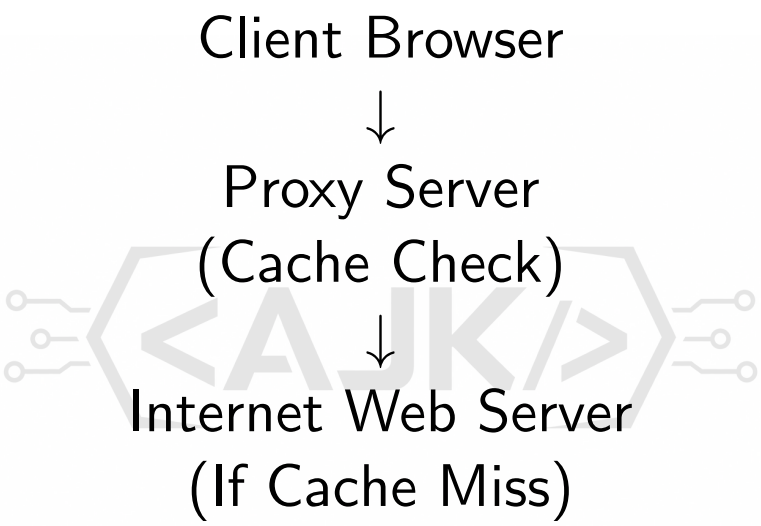
Benefits of Proxy Caching

- Reduces:
  - Internet bandwidth usage
  - Traffic load
- Improves:
  - Access speed
  - Response time
- Frequently requested pages:
  - Retrieved locally
- Decreases load on:
  - Original web server

Proxy Server Communication Flow

1. Client browser sends HTTP request
2. Proxy server checks cache
3. If cached:
   - Send page to client
4. If not cached:
   - Proxy requests page from Internet server
5. Web server sends response
6. Proxy stores copy in cache
7. Proxy forwards response to client

Illustration of Example 2.10



- Proxy server acts between clients and Internet

Key Points from Example 2.10

- Proxy server installed in:
  - Local network
- Proxy stores:
  - Copies of requested web pages
- Future requests served from:
  - Local cache
- Results:
  - Faster access
  - Reduced Internet traffic
  - Lower server load

Cache Update

- Important issue in proxy caching:
  - How long cached response should remain stored
- Cached data eventually:
  - Deleted or replaced
- Different cache update strategies:
  - Based on type of website
  - Based on modification time
- Example:
  - News websites update content daily
  - Proxy may cache news page for one day
- HTTP headers may contain:
  - Last modification time
- Proxy server uses header information to:
  - Estimate validity period of cached data

HTTP Security and HTTPS

- HTTP itself:
  - Does not provide security
- Security achieved using:
  - Secure Socket Layer (SSL)
- HTTP running over SSL called:
  - HTTPS
- HTTPS provides:
  - Confidentiality
  - Client authentication
  - Server authentication
  - Data integrity
- HTTPS used for:
  - Secure web communication
  - Online banking
  - E-commerce transactions